



A HANDS-ON GUIDE

JavaScript Crash Course



From variables to async/await — learn modern JavaScript by writing real code, one small chapter at a time.

Edition 2026 · Covers ES2015–ES2024 · Includes exercises & answers

Table of Contents

01	Getting Started	1
02	Variables & Data Types	3
03	Operators & Control Flow	6
04	Functions	9
05	Arrays	12
06	Objects	15
07	The DOM & Events	18
08	Asynchronous JavaScript	21
09	ES6+ Modern Features	24
10	Error Handling & Debugging	27
11	Classes & OOP	29
12	Where To Go Next	32

Getting Started

JavaScript is the programming language of the web. It runs in every browser, powers servers through Node.js, and drives mobile apps, desktop apps, and even embedded devices. Unlike most first languages, you don't need to install anything to try it — every browser already has a JavaScript engine built in.

Your First Line of Code

Open any browser, press **F12** (or **Cmd+Opt+J** on Mac) to open Developer Tools, click the **Console** tab, and type:

CONSOLE

```
console.log("Hello, JavaScript!");
```

OUTPUT

Hello, JavaScript!

Congratulations — you just executed JavaScript. `console.log()` is the single most useful function you'll use while learning: it prints values so you can see what your code is doing.

Running JavaScript in a File

In real projects, code lives in `.js` files, linked from HTML:

INDEX.HTML

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My Page</h1>
    <script src="app.js"></script>
  </body>
</html>
```

APP.JS

```
console.log("app.js is running!");
```

Running JavaScript with Node.js

Outside the browser, [Node.js](#) lets you run JavaScript directly from the terminal:

TERMINAL

save a file called hello.js, then run:

\$ node hello.js

Hello, JavaScript!

NOTE

Every code example in this book can be pasted straight into a browser console or a Node.js file — try them as you read instead of just reading them.

Comments

Comments are ignored by the JavaScript engine. Use them to explain *why*, not *what*:

// This is a single-line comment

*/**

This is a

multi-line comment

**/*

let price = 19.99; *// price in USD*

Chapter Summary

- JavaScript runs in browsers (via Developer Tools or `<script>` tags) and on servers via Node.js.
- `console.log()` prints values — your main debugging tool.
- Comments (`//` and `/* */`) document code without affecting execution.

CHAPTER 02

Variables & Data Types

Variables store data so your program can use it later. Modern JavaScript gives you three ways to declare one.

let, const, and var

```
let age = 25;    // can be reassigned
const name = "Ava"; // cannot be reassigned
var legacy = "old style"; // avoid in modern code

age = 26;    // OK
name = "Zoe"; // ❌ TypeError: Assignment to constant variable.
```

Keyword	Reassignable	Scope	Use it when...
<code>const</code>	No	Block	Default choice for everything
<code>let</code>	Yes	Block	The value needs to change
<code>var</code>	Yes	Function	Almost never (legacy code only)

TIP

Default to `const`. Only switch to `let` when you know the variable's value must change.

Primitive Data Types

```
let str = "Hello";    // String
let num = 42;         // Number
let big = 9007199254740993n; // BigInt
let isDone = true;    // Boolean
let nothing = null;   // null (intentional empty value)
let notSet = undefined; // undefined (not yet assigned)
let id = Symbol("id"); // Symbol (unique identifier)

typeof str;    // "string"
typeof num;    // "number"
typeof isDone; // "boolean"
```

Template Literals

Backticks let you embed expressions directly inside strings:

```
const user = "Sam";
const cartTotal = 42.5;

console.log(`Hi ${user}, your total is ${cartTotal.toFixed(2)}`);
```

OUTPUT

Hi Sam, your total is \$42.50

Type Coercion

JavaScript will automatically convert types in some operations — a frequent source of bugs:

```
"5" + 3; // "53" (number becomes a string)
"5" - 3; // 2 (string becomes a number)
5 + true; // 6 (true becomes 1)
5 == "5"; // true (loose equality coerces types)
5 === "5"; // false (strict equality checks type too)
```

WARNING

Always prefer `===` and `!==` over `==` and `!=`. Loose equality's coercion rules are a common source of bugs.

Chapter Summary

- Use `const` by default, `let` when reassignment is needed, and avoid `var`.
- Primitive types: string, number, bigint, boolean, null, undefined, symbol.
- Template literals (``${}``) embed expressions in strings.
- Use `===` / `!==` to avoid surprise type coercion.

Try It Yourself

1. Declare a `const` named `pi` set to `3.14159`.
2. Declare a `let` named `radius` set to `5`.
3. Use a template literal to log: `"The area is 78.54"` (area = $\pi \times r^2$).

Operators & Control Flow

Arithmetic & Comparison Operators

```
10 + 3; // 13
10 % 3; // 1 (remainder)
2 ** 10; // 1024 (exponent)
10 > 3; // true
10 <= 9; // false
```

Logical Operators

```
true && false; // false — AND
true || false; // true — OR
!true; // false — NOT

const username = null;
const displayName = username ?? "Guest"; // "Guest" — nullish coalescing
```

if / else if / else

```
const temp = 28;

if (temp > 30) {
  console.log("It's hot");
} else if (temp > 20) {
  console.log("It's warm"); // this runs
} else {
  console.log("It's cold");
}
```

switch

```
const day = "MON";

switch (day) {
  case "SAT":
  case "SUN":
    console.log("Weekend");
    break;
  default:
    console.log("Weekday"); // this runs
}
```

Loops

```
// for loop
for (let i = 0; i < 3; i++) {
  console.log(`Iteration ${i}`);
}
// Iteration 0
// Iteration 1
// Iteration 2

// while loop
let count = 3;
while (count > 0) {
  console.log(count);
  count--;
}

// for...of — iterate values
for (const fruit of ["apple", "pear"]) {
  console.log(fruit);
}
```

Chapter Summary

- `&&`, `||`, and `??` handle logical branching and defaults.
- `if/else` and `switch` control which code runs.
- `for`, `while`, and `for...of` repeat code.

CHAPTER 04

Functions

Functions are reusable blocks of code. JavaScript offers three main ways to write one.

```
// 1. Function declaration
function add(a, b) {
  return a + b;
}

// 2. Function expression
const subtract = function(a, b) {
  return a - b;
};

// 3. Arrow function (modern, concise)
const multiply = (a, b) => a * b;

add(2, 3);    // 5
subtract(5, 2); // 3
multiply(4, 3); // 12
```

Default Parameters & Rest

```
function greet(name = "stranger") {
  return `Hello, ${name}!`;
}

greet();    // "Hello, stranger!"
greet("Priya"); // "Hello, Priya!"

function sumAll(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

sumAll(1, 2, 3, 4); // 10
```

Arrow Functions & `this`

Arrow functions don't have their own `this` — they inherit it from the surrounding scope. This makes them ideal for callbacks:

```
class Timer {
  seconds = 0;

  start() {
    setInterval(() => {
      this.seconds++; // `this` correctly refers to the Timer instance
    }, 1000);
  }
}
```

Higher-Order Functions

Functions that take or return other functions:

```
function makeMultiplier(factor) {
  return (n) => n * factor;
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);

double(5); // 10
triple(5); // 15
```

NOTE

This pattern — a function returning a function that "remembers" surrounding variables — is called a **closure**. Closures power hooks, event handlers, and memoization.

Try It Yourself

1. Write an arrow function `isEven(n)` that returns `true` if a number is even.
2. Write a function `makeCounter()` that returns a function which increments and returns a count each time it's called.

CHAPTER 05

Arrays

Arrays store ordered lists of values and come with powerful built-in methods.

```
const fruits = ["apple", "banana", "cherry"];

fruits[0];    // "apple"
fruits.length; // 3
fruits.push("date"); // adds to end → ["apple","banana","cherry","date"]
fruits.pop();  // removes & returns last item → "date"
```

The Big Three: map, filter, reduce

```
const nums = [1, 2, 3, 4, 5];

// map: transform each item into a new array
const doubled = nums.map(n => n * 2);
// [2, 4, 6, 8, 10]

// filter: keep items that pass a test
const evens = nums.filter(n => n % 2 === 0);
// [2, 4]

// reduce: fold the array down to a single value
const total = nums.reduce((sum, n) => sum + n, 0);
// 15
```

Method	Purpose	Returns
<code>.map()</code>	Transform each element	New array (same length)
<code>.filter()</code>	Keep matching elements	New array ($\leq$ length)
<code>.reduce()</code>	Combine into one value	Any single value
<code>.find()</code>	First match	One element or <code>undefined</code>
<code>.some()</code> / <code>.every()</code>	Test elements	Boolean
<code>.forEach()</code>	Run side-effects	<code>undefined</code>

Destructuring & Spread

```
const [first, second, ...rest] = ["a", "b", "c", "d"];  
// first = "a", second = "b", rest = ["c", "d"]  
  
const combined = [...fruits, "elderberry"];  
// spreads fruits into a new array, then appends one more
```

WARNING

`.map()`, `.filter()`, and spread all return *new* arrays — the original is untouched. This "immutability by default" mindset avoids subtle bugs.

Try It Yourself

1. Given `const prices = [10, 25, 4, 99, 12]`, use `.filter()` to get prices under 20.
2. Use `.reduce()` to find the total of all prices.
3. Use `.map()` to convert each price to a string like `"$10"`.

CHAPTER 06

Objects

Objects store key-value pairs and model real-world entities.

```
const user = {  
  name: "Maria",  
  age: 29,  
  isAdmin: false,  
  greet() {  
    return `Hi, I'm ${this.name}`;  
  }  
};  
  
user.name;    // "Maria"    — dot notation  
user["age"];  // 29         — bracket notation  
user.greet(); // "Hi, I'm Maria"
```

Destructuring, Spread & Optional Chaining

```
const { name, age } = user;  
// name = "Maria", age = 29  
  
const updated = { ...user, age: 30 };  
// copies user, then overrides age → { name: "Maria", age: 30, ... }  
  
const city = user.address?.city ?? "Unknown";  
// safely reads a nested property that might not exist
```

Iterating Objects

```
Object.keys(user); // ["name", "age", "isAdmin", "greet"]  
Object.values(user); // ["Maria", 29, false, f]  
Object.entries(user); // [["name", "Maria"], ["age", 29], ...]  
  
for (const [key, value] of Object.entries(user)) {  
  console.log(`${key}: ${value}`);  
}
```

Chapter Summary

- Objects group related data and behavior using key-value pairs.
- Destructuring and spread (`{...obj}`) make copying and extracting concise.
- Optional chaining (`?.`) and nullish coalescing (`??`) guard against missing data.

CHAPTER 07

The DOM & Events

The Document Object Model (DOM) is how JavaScript reads and manipulates the page.

```
// Selecting elements
const heading = document.querySelector("h1");
const buttons = document.querySelectorAll(".btn");

// Changing content & style
heading.textContent = "Welcome back!";
heading.style.color = "purple";
heading.classList.add("highlight");

// Creating elements
const item = document.createElement("li");
item.textContent = "New item";
document.querySelector("ul").appendChild(item);
```

Handling Events

```
const button = document.querySelector("#save-btn");

button.addEventListener("click", (event) => {
  console.log("Button was clicked!", event.target);
});

// Form submission
document.querySelector("form").addEventListener("submit", (e) => {
  e.preventDefault(); // stop the page from reloading
  const data = new FormData(e.target);
  console.log(data.get("email"));
});
```

TIP

Always call `e.preventDefault()` inside a form's `submit` handler if you're handling the data with JavaScript instead of a normal page reload.

Try It Yourself

1. Create a button that, when clicked, adds a new `<li>` to a list with the current timestamp.
2. Add an input field and a "counter" that displays how many characters have been typed, updating on every keystroke (`"input"` event).

CHAPTER 08

Asynchronous JavaScript

JavaScript is single-threaded, but async patterns let it handle slow operations — like network requests — without freezing the page.

Callbacks (the old way)

```
setTimeout(() => {  
  console.log("3 seconds have passed");  
}, 3000);
```

Promises

```
function fetchUser(id) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      if (id > 0) {  
        resolve({ id, name: "Leo" });  
      } else {  
        reject(new Error("Invalid ID"));  
      }  
    }, 1000);  
  });  
}  
  
fetchUser(1)  
  .then(user => console.log(user))  
  .catch(err => console.error(err))  
  .finally(() => console.log("Done"));
```

async / await (the modern way)

Syntactic sugar over Promises that reads like synchronous code:

```
async function loadUser() {
  try {
    const user = await fetchUser(1);
    console.log(user); // { id: 1, name: "Leo" }
  } catch (error) {
    console.error("Failed:", error.message);
  }
}

loadUser();
```

Real-World Example: fetch()

```
async function getPosts() {
  const response = await fetch("https://api.example.com/posts");
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const posts = await response.json();
  return posts;
}
```

Running Things in Parallel

```
const [user, posts, comments] = await Promise.all([
  fetchUser(1),
  fetchPosts(1),
  fetchComments(1)
]);
// all three run concurrently instead of one after another
```

NOTE

`await` can only be used inside a function marked `async` (or at the top level of a module).

Chapter Summary

- Promises represent a value that will exist eventually — pending, fulfilled, or rejected.
- `async/await` makes asynchronous code read like synchronous code.
- `Promise.all()` runs independent async tasks concurrently.

ES6+ Modern Features

Modules

```
// math.js
export function add(a, b) { return a + b; }
export const PI = 3.14159;
export default function multiply(a, b) { return a * b; }

// app.js
import multiply, { add, PI } from './math.js';
```

Sets & Maps

```
const uniqueIds = new Set([1, 2, 2, 3]);
// Set(3) { 1, 2, 3 } — duplicates automatically removed

const scores = new Map();
scores.set("Alice", 95);
scores.set("Bob", 88);
scores.get("Alice"); // 95
```

Optional Chaining, Nullish Coalescing & Destructuring Recap

```
const config = { db: { host: "localhost" } };

const port = config.db?.port ?? 5432; // 5432 (not set, so fallback used)

function connect({ host, port = 5432 }) {
  return `Connecting to ${host}:${port}`;
}
```

Array & Object Spread in Practice

```
const defaults = { theme: "light", fontSize: 14 };
const userPrefs = { fontSize: 18 };

const settings = { ...defaults, ...userPrefs };
// { theme: "light", fontSize: 18 } — later spreads win
```

Chapter Summary

- ES modules (`import` / `export`) organize code across files.
- `Set` stores unique values; `Map` stores key-value pairs with any key type.
- Spread and destructuring simplify merging and extracting data.

Error Handling & Debugging

try / catch / finally

```
function parseConfig(json) {  
  try {  
    return JSON.parse(json);  
  } catch (error) {  
    console.error("Invalid config:", error.message);  
    return {};  
  } finally {  
    console.log("Parse attempt finished");  
  }  
}
```

Throwing Custom Errors

```
class ValidationError extends Error {  
  constructor(message) {  
    super(message);  
    this.name = "ValidationError";  
  }  
}  
  
function setAge(age) {  
  if (age < 0) {  
    throw new ValidationError("Age cannot be negative");  
  }  
  return age;  
}  
  
try {  
  setAge(-5);  
} catch (e) {  
  console.log(e.name, "-", e.message);  
  // ValidationError - Age cannot be negative  
}
```

Debugging Techniques

```
console.table([ { id: 1, name: "A" }, { id: 2, name: "B" } ]);  
console.warn("Deprecated function used");  
console.error("Something broke");
```

```
debugger; // pauses execution when DevTools is open
```

TIP

Use `console.table()` to inspect arrays of objects — it's far more readable than `console.log()` for tabular data.

Try It Yourself

1. Write a function `divide(a, b)` that throws an `Error` when `b` is 0, and catch it with `try/catch`.
2. Create a custom `NotFoundError` class extending `Error`.

CHAPTER 11

Classes & OOP

Classes are syntactic sugar over JavaScript's prototype-based inheritance, offering a familiar structure for modeling objects.

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  speak() {
    return `${this.name} says ${this.sound}`;
  }
}

const dog = new Animal("Rex", "Woof");
dog.speak(); // "Rex says Woof"
```

Inheritance

```
class Dog extends Animal {
  constructor(name, breed) {
    super(name, "Woof"); // call the parent constructor
    this.breed = breed;
  }

  fetch() {
    return `${this.name} fetches the ball!`;
  }
}

const rex = new Dog("Rex", "Labrador");
rex.speak(); // inherited: "Rex says Woof"
rex.fetch(); // own method: "Rex fetches the ball!"
```

Getters, Setters & Private Fields

```

class BankAccount {
  #balance = 0; // private field — inaccessible outside the class

  deposit(amount) {
    this.#balance += amount;
  }

  get balance() {
    return `${this.#balance.toFixed(2)}`;
  }
}

const account = new BankAccount();
account.deposit(100);
account.balance; // "$100.00" (via getter)
account.#balance; // ❌ SyntaxError: Private field outside class

```

Static Members

```

class MathUtils {
  static square(n) {
    return n * n;
  }
}

MathUtils.square(5); // 25 — called on the class, not an instance

```

Chapter Summary

- `class` and `extends` model inheritance hierarchies.
- `#field` syntax creates true private state.
- `get` / `set` let you expose computed or validated properties.
- `static` members belong to the class itself, not instances.

Try It Yourself

1. Create a `Shape` class with a `area()` method returning 0.
2. Create `Circle` and `Rectangle` classes that extend `Shape` and override `area()`.
3. Add a private `#id` field and a getter to read it.

Where To Go Next

You now know enough core JavaScript to build real things. Here's a map for continuing.

Practice Ideas

- Build a to-do list app using the DOM chapter's techniques.
- Build a weather app using `fetch()` against a public API.
- Recreate a simple quiz game using arrays, objects, and event listeners.

Where JavaScript Goes From Here

Topic	What it's for
TypeScript	Adds static types on top of JavaScript
React / Vue / Svelte	Frameworks for building UIs
Node.js + Express	Building servers and APIs
Jest / Vitest	Automated testing
Webpack / Vite	Bundling and build tooling

NOTE

Don't try to learn everything at once. Pick one small project, use only what you need, and let new concepts enter as real problems demand them.

Final Challenge

```
// Combine everything from this book:  
// classes, arrays, async/await, destructuring, and the DOM.
```

```
class TodoApp {  
  #todos = [];  
  
  add(text) {  
    this.#todos = [...this.#todos, { text, done: false }];  
  }  
  
  complete(index) {  
    this.#todos = this.#todos.map((todo, i) =>  
      i === index ? { ...todo, done: true } : todo  
    );  
  }  
  
  get pending() {  
    return this.#todos.filter(t => !t.done);  
  }  
}
```

```
const app = new TodoApp();  
app.add("Learn JavaScript");  
app.add("Build a project");  
app.complete(0);  
console.log(app.pending);  
// [{ text: "Build a project", done: false }]
```

TIP

Keep this book as a reference. The best way to solidify what you've learned is to build something you actually want to use.